

Renova

DESIGN DOCUMENT

sdmay26-16

Advisor MD Maruf Ahamed

Team Members & Roles:

Prakeerth Regunath: Frontend

Onur Onal: Project Lead & Backend

Kemal Yavuz: Full-stack

Erroll Barker: Frontend

Bae Meh: Backend

Om Sanghvi: Backend

Team Email

sdmay26-16@iastate.edu

Team Website

<https://sdmay26-16.sd.ece.iastate.edu/>

Revised: 9/30/2025

Executive Summary

Renova is a web-based platform designed to combine user creativity with the power of artificial intelligence. The core problem we address is the high cost, time commitment, and expertise barrier associated with professional interior design services. Many homeowners and renters struggle to visualize renovation ideas before committing financially, leading to dissatisfaction and wasted resources. Renova makes this process accessible, intuitive, and risk-free.

Our key design requirements are to create a user-friendly moodboard editor for arranging room items, a robust backend to manage user data and AI job orchestration, and a specialized microservice for generating precise image masks and depth maps. The platform must generate high-quality, realistic previews of renovated spaces based on the user's unique design choices.

The design is decomposed into three interconnected subsystems: a Frontend built with Next.js for the interactive moodboard, a Backend API using Node.js and MySQL for data management, and a Python-based ML Microservice leveraging PyTorch models (MiDaS, SAM2) for image preprocessing. The final AI image generation is handled by the Replicate API (Stable Diffusion XL).

Progress thus far is in the detailed planning and architectural design phase. We have a clear roadmap, technology stack, and system architecture defined. The design effectively meets user needs by providing a creative canvas for ideation and an AI-powered “magic wand” to see those ideas come to life. This is validated by the direct mapping of features (e.g., drag-and-drop, AI previews) to user goals (visualization, cost-saving).

The next steps involve implementing the Minimum Viable Product (MVP) in Semester 1, focusing on core functionality like basic canvas operations, user authentication, and integrating the depth map generation service.

Learning Summary

Development Standards & Practices Used

Software Development: Agile methodology, Version Control (Git), RESTful API design, Modular code structure.

Frontend: Component-based architecture (Next.js), Responsive web design principles.

Backend: Secure Authentication (JWT), Database Normalization (MySQL), API Endpoint security.

Machine Learning: Use of pre-trained models (MiDaS, SAM2) following PyTorch best practices, API encapsulation with FastAPI.

Summary of Requirements

Functional: User registration/login, Image upload to a personal library, Drag-and-drop moodboard editor (resize, rotate, layer items), AI-powered renovation preview generation, Side-by-side comparison of results.

Non-Functional: Responsive and intuitive user interface, Secure handling of user data, Scalable backend architecture, Reasonable processing time for AI generation (< 2 minutes).

Applicable Courses from Iowa State University Curriculum

- COM S 309: Software Development Practices
- SE 319: Construction of User Interfaces
- COM S 228: Introduction to Data Structures
- COM S 339: Software Architecture
- COM S 363: Introduction to Database Management Systems
- COM S 417: Software Testing
- Various Machine Learning / Data Science courses (for the ML microservice)

New Skills/Knowledge acquired that were not taught in courses

- Integration of multiple AI models (MiDaS, SAM2, Stable Diffusion) into a single pipeline.
- Specifics of the Replicate API and inpainting/depth-guided image generation.
- Using the react-konva and dnd-kit libraries for advanced canvas interactions.
- Orchestrating a microservices architecture with communication between Node.js and Python/FastAPI services.

Table of Contents

1. Introduction	7
2. Requirements, Constraints, And Standards	7
3 Project Plan	8
3.1 Project Management/Tracking Procedures	8
3.2 Task Decomposition	8
3.3 Project Proposed Milestones, Metrics, and Evaluation Criteria	9
3.4 Project Schedule	9
3.5 Risks and Risk Management/Mitigation	11
3.6 Personnel Effort Requirements	12
4 Design	12
4.1 Design Context	12
4.1.1 Broader Context	12
4.1.2 Prior Work/Solutions	13
4.1.3 Technical Complexity	13
4.2 Design Exploration	14
4.2.1 Design Decisions	14
4.2.2 Ideation	14
4.2.3 Decision-Making and Trade-Off	14
4.3 Proposed Design	15
4.3.1 Overview	15
4.3.2 Detailed Design and Visual(s)	15
Subsystem Overview	17
Integration and Data Flow	18
4.3.3 Functionality	18
4.3.4 Areas of Concern and Development	19
4.4 Technology Considerations	19
4.5 Design Analysis	20
5 Testing	20
5.1 Unit Testing	20
5.2 Interface Testing	20
5.3 Integration Testing	21
5.4 System Testing	21
5.5 Regression Testing	21
5.6 Acceptance Testing	21
5.7 Security Testing (if applicable)	21
5.8 User Testing	21
5.9 Results	21
6 Implementation	22
7 Ethics and Professional Responsibility	22
7.1 Areas of Professional Responsibility/Codes of Ethics	22
7.2 Four Principles	23
7.3 Virtues	24
8 Closing Material	26
8.1 Conclusion	26
8.2 References	26
8.3 Appendices	27
9 Team	28
9.6 Team Contract	29

List of figures/tables/symbols/definitions

Figures

- **Figure 4.3.2.1** - Renova System Block Diagram. Shows the interaction between the frontend, backend, ML microservice, external image generation service, MySQL database, and object storage.

Tables

- **Table 3.4.1** - Project Schedule (Semester 1 & 2 Sprint Breakdown). Provides a sprint-by-sprint breakdown of frontend, backend, and ML service tasks, along with integrated deliverables.
- **Table 3.6.1** - Personnel Effort Requirements. Estimates the number of hours required for major tasks, including frontend, backend, and ML service development.
- **Table 4.1.1.1** - Broader Context Considerations. Summarizes impacts of the project in public health, global/cultural/social, environmental, and economic areas, with examples.
- **Table 4.2.3.1** - Decision-Making and Trade-Off Matrix. Compares multiple frontend canvas implementation options and highlights the rationale for selecting React-Konva.

Symbols

- **/render** - Core backend API endpoint that orchestrates renovation jobs.
- **IoU (Intersection over Union)** - Metric used to evaluate segmentation accuracy.
- **MVP (Minimum Viable Product)** - The baseline version of the system targeted for delivery at the end of Semester 1.

Definitions

- **Moodboard Canvas** - An Interactive editor where users arrange furniture and décor assets over a room photo.
- **Segmentation Mask** - Output from SAM2 used to identify and isolate objects in a room photo.
- **Depth Map** - Output from MiDaS showing relative distance of objects from the camera, used to guide AI rendering.

- **Signed URL** - Secure, time-limited link granting users access to files stored in object storage.

1. Introduction

1.1. PROBLEM STATEMENT

Imagine you want to redecorate your living room. You have ideas: a new sofa, different colored walls, and a modern lamp. However, you can't picture how it will all look together. Hiring an interior designer is expensive, and buying furniture only to discover it doesn't fit the space is a costly mistake. This is a common frustration for homeowners and renters. The problem is the gap between imagination and reality in home renovation. Renova addresses this by providing a digital sandbox where users can visually plan their space and, crucially, see a photorealistic AI-generated preview of their ideas applied to their actual room. This reduces financial risk, boosts confidence in design decisions, and makes interior design accessible to everyone, not just professionals.

1.2. INTENDED USERS

The DIY Homeowner: Tech-savvy individuals who enjoy home improvement projects but lack formal design training. They need a tool to experiment with styles and visualize changes without commitment. Renova provides them with a risk-free environment to plan and perfect their renovations.

The First-Time Renter: Young adults living in rental properties who want to personalize their space within landlord constraints. Their need is for non-permanent, reversible design ideas. Renova helps them visualize temporary changes like new furniture and decor, ensuring they love the result before purchasing anything.

The Interior Design Student: Students learning design principles who need a rapid prototyping tool to test concepts and build portfolios. They benefit from Renova's quick iteration cycle, allowing them to explore multiple design variations for a single room efficiently.

2. Requirements, Constraints, And Standards

2.1. REQUIREMENTS & CONSTRAINTS

- **User Account Management:** The system shall allow users to create an account and log in. **(Functional)**
- **Asset Library:** The system shall provide a "My Uploads" library for users to store images of items and rooms. **(Functional)**
- **Moodboard Editor:** The system shall provide a canvas where users can drag, drop, resize, rotate, and layer images from their library. **(Functional, User Experience)**
- **AI Renovation Trigger:** The system shall generate a renovation preview when a user clicks a "Renovate" button. **(Functional)**

- **Preview Display:** The system shall display generated previews in a gallery with comparison tools (e.g., side-by-side, slider). **(Functional)**
- **Performance:** The AI preview generation process shall complete within 120 seconds for a standard room image. **(Constraint)**
- **Data Security:** User passwords shall be hashed before storage in the database. **(Constraint)**

2.2. ENGINEERING STANDARDS

Importance of Standards: Engineering standards ensure consistency, interoperability, and safety across systems. They provide a common framework that allows software and hardware components to integrate smoothly, while also protecting user data and ensuring quality. For a project like Renova, standards help the team adopt industry best practices in security, software engineering, and data handling.

Relevant Standards:

- **ISO/IEC 27001 (Information Security Management):** Ensures secure handling of user data, aligning with our requirement to hash and protect user passwords.
- **IEEE 830 (Software Requirements Specification):** Guides the documentation of functional and non-functional requirements, ensuring clarity and completeness in system design.
- **ISO/IEC 25010 (System and Software Quality Models):** Provides a framework for evaluating software quality (usability, performance, reliability), which directly applies to our functional and non-functional requirements.

Relevance to Project: These standards are directly applicable because they address secure data management, clear documentation of requirements, and measurable quality benchmarks, all essential for Renova's success.

Modifications to Design for Standards:

- Adopt secure password handling and encrypted communication in line with ISO/IEC 27001.
- Formalize requirements documentation following IEEE 830, ensuring traceability to user needs.
- Use ISO/IEC 25010 as a reference when evaluating usability and performance during testing.

3 Project Plan

3.1 PROJECT MANAGEMENT/TRACKING PROCEDURES

We are adopting an Agile methodology with elements of Scrum. This is justified because our project involves iterative development of complex, interdependent features (e.g., moodboard, AI pipeline). Agile allows us to adapt to challenges, such as integrating different AI models, and deliver a working MVP quickly.

Tracking tools: Gitlab (code repository, issue tracking), Discord/ Weekly In-Person Team Meetings (Communication).

3.2 TASK DECOMPOSITION

The project is decomposed into three main task streams, mirroring our architecture:

- 1. **Frontend (Next.js) Stream:**
 - a. User Authentication UI
 - b. Asset Library UI
 - c. Moodboard Canvas Component
 - d. Preview Gallery UI
- 2. **Backend (Node.js) Stream:**
 - a. User Authentication API
 - b. CRUD APIs for Projects/Boards/Assets
 - c. /render Job Orchestration Endpoint
 - d. Database Schema & Integration
- 3. **ML Microservice (Python) Stream:**
 - a. FastAPI Server Setup
 - b. MiDaS Depth Model Integration
 - c. SAM2 Segmentation Model Integration

3.3 PROJECT PROPOSED MILESTONES, METRICS, AND EVALUATION CRITERIA

- Milestone 1 (End of Semester 1 - MVP): A user can log in, create and use an interactive moodboard, upload a room image, and receive a basic AI-generated renovation preview.
 - Metric: Working moodboard and successful end-to-end generation of a preview image via the Replicate API.
- Milestone 2 (Mid Semester 2): Moodboard features are fully developed (undo/redo, snap-to-grid), and the ML microservice automatically generates depth maps and masks.
 - Metric: Mask generation accuracy measured by IoU (Intersection over Union) against manual masks.
- Milestone 3 (End of Semester 2): Full integration of all components, performance optimization, and user testing.
 - Metric: User satisfaction score of 4/5 or higher on a usability survey.

3.4 PROJECT SCHEDULE

Detailed Sprint Schedule & Task Breakdown (Semesters 1 & 2)

Semester 1: MVP (Minimum Viable Product) Foundation

Sprint	Timeframe	Frontend Tasks	Backend Tasks	ML Tasks	Integrated Deliverables
--------	-----------	----------------	---------------	----------	-------------------------

1-2	Early Sept - End Oct	- Set up Next.js app. - Create Login/Signup Components. - Build basic dashboard layout.	- Set up Express.js server. - Design MySQL schema (Users, Projects). - Implement JWT authentication API.	- Research Replicate API (SDXL). - Document API call parameters for inpainting.	Basic authenticated application skeleton.
3-4	Early Nov- Mid Nov	- Develop "My Uploads" page. - Implement image uploader component & gallery view.	- Build Asset CRUD APIs (upload, list, delete). - Set up cloud storage for images.	- Create a proof-of-concept script: call the Replicate API with a test image and prompt.	Users can upload and manage their image library.
5-6	Mid Nov- End Nov	- Implement core Moodboard Canvas with React-Konva. - Enable drag, drop, and resize of images from the library onto the canvas.	- Create Project/Board CRUD APIs. - Develop an endpoint to save/load canvas JSON state.	- Formalize the "job" data structure needed from the frontend (image URLs, mask data, prompt). - Build a helper module to construct Replicate API calls.	Functional moodboard editor that saves/loads state.
7-8	Early Dec - Mid Dec	- Build Preview Gallery UI for side-by-side comparison. - Connect "Renovate" button to backend endpoint. - Polish full user flow.	- Build the /render job orchestration endpoint. - Integrate Replicate API module; handle async job polling and status updates. - Store the final generated images.	- Develop basic error handling for AI generation failures.	Milestone 1: MVP Demo End-to-End Flow: Moodboard -> AI Preview -> Comparison.

Semester 2: Enhancement & Integration

Sprint	Timeframe	Frontend Tasks	Backend Tasks	ML Tasks	Integrated Deliverables
9-10	Mid Jan- End Jan	- Add moodboard features: rotate, layer ordering (z-index).	- Refactor backend to call the new ML	- Set up FastAPI server.	Enhanced moodboard.

		- Implement undo/redo functionality.	- service. - Create new endpoints: /generate-depth and /generate-mask. - Update data model for new pipeline.	- Integrate the MiDaS model; create a depth map generation endpoint. - Dockerize the service.	Functional depth map API.
11-12	Early Feb- Mid Feb	- Develop a UI for manual mask refinement (contingency for Risk 2). - Add visual feedback for the generation pipeline status.	- Integrate the SAM2 segmentation endpoint from the ML service. - Update the /render pipeline: Backend -> ML Service (depth/mask) -> Replicate API.	- Integrate the SAM2 model; create a segmentation endpoint. - Combine depth & mask generation into a unified preprocessing pipeline.	Milestone 2: Automated Masking Full AI pipeline using the custom ML service.
13-14	Mid Feb- Early Mar	- Performance optimization (e.g., virtual scrolling in asset library). - Conduct internal usability tests and gather feedback.	- Orchestrate and optimize the full, multi-step pipeline. - Implement caching for ML results to reduce costs and latency. - Conduct backend load testing..	- Optimize model inference speed (e.g., model quantization). - Improve mask accuracy and handle edge cases from user tests.	Fully integrated, optimized, and stable AI renovation pipeline.
15-16	Early Apr- End Apr	- Bug fixing, UI/UX polish, and final responsiveness checks. - Conduct formal user testing with target users.	- Final security review and API hardening. - Prepare for deployment (environment variables, configs).	- Final model tuning based on user test data. - Prepare ML service for production deployment.	Milestone 3: Final Delivery A polished, tested, and deployable product.

3.5 RISKS AND RISK MANAGEMENT/MITIGATION

- Risk 1: The Replicate API cost exceeds the ideal budget with high usage.
 - Probability: Medium

- Mitigation: Implement usage quotas, cache results, and research alternative/self-hosted models.
- Risk 2: The SAM2 model does not segment furniture with sufficient accuracy.
 - Probability: High
 - Mitigation: Plan for a manual mask refinement tool in the UI; explore fine-tuning the model on interior design datasets.
- Risk 3: Performance lag in the React-Konva canvas with many items.
 - Probability: Medium
 - Mitigation: Implement virtual scrolling for the asset library and optimize canvas redraw logic.

3.6 PERSONNEL EFFORT REQUIREMENTS

Task	Estimated Hours
Frontend: Moodboard Canvas	80
Backend: Auth & CRUD APIs	60
ML Service:	40
Total	~180

4 Design

4.1 DESIGN CONTEXT

4.1.1 Broader Context

Renova is situated within the communities of homeowners, renters, and design enthusiasts. It addresses the societal need for affordable and accessible creative tools, empowering individuals to improve their personal living environments without the high cost of professional services.

Area	Description	Examples
Public health, safety, and welfare	Renova improves welfare by reducing the financial risk and stress associated with home renovation. It promotes satisfaction and well-being by helping users create living spaces they love.	A user planning a renovation can preview design changes virtually before committing money, reducing the stress of costly mistakes.
Global, cultural, and social	The platform must be designed to be inclusive of diverse aesthetic tastes and cultural styles. The AI models need to be trained on diverse datasets to avoid bias and generate culturally relevant designs.	The platform includes templates inspired by Japanese minimalism, Mediterranean styles, and modern urban aesthetics so that users from different cultures feel represented.

Environmental	The project is digital and has a minimal direct environmental footprint. It can indirectly promote sustainability by allowing users to "try before they buy," reducing wasteful purchases of physical items that don't work in a space.	A renter uses the tool to test multiple wall paint options digitally instead of buying several paint samples, minimizing waste.
Economic	Renova provides an economical alternative to hiring interior designers. The challenge is to keep the service cost-effective for users, which involves managing the costs of cloud-based AI APIs.	A first-time homeowner can try various furnishing options using affordable subscription-based access, reducing the need for expensive professional consultations.

4.1.2 Prior Work/Solutions

Several digital tools exist for moodboarding and AI-assisted interior design, but none fully integrate both in the way Renova proposes.

- **Morpholio Board** is a professional-grade moodboard app used by interior designers. It allows users to arrange products and create polished presentations, but it lacks AI-driven renovation previews tied to a user's actual room image [1].
- **Canva** offers a general-purpose drag-and-drop design editor, widely used for creative projects. While accessible and easy to use, Canva is not tailored to interior design and does not incorporate spatial awareness or AI rendering for real environments [2].
- **RoomGPT.io** is an AI tool that generates alternative room styles from a single uploaded photo. However, it does not provide an interactive moodboard editor for iterative design, limiting user creativity and control [3].

Compared to these, Renova uniquely combines an interactive moodboard editor with an AI renovation pipeline. This hybrid approach gives users both creative control (through arranging assets) and automation (through photorealistic previews). The trade-off is increased technical complexity, balancing UI performance, ML model accuracy, and API costs, but the result is a more versatile and user-centered solution.

References

- [1] Morpholio, "Board App," [Online]. Available: <https://www.morpholioapps.com/board>. [Accessed: 30-Sep-2025].
- [2] Canva, "Online Design Platform," [Online]. Available: <https://www.canva.com>. [Accessed: 30-Sep-2025].
- [3] RoomGPT, "AI Room Redesign," [Online]. Available: <https://www.roomgpt.io>. [Accessed: 30-Sep-2025].

4.1.3 Technical Complexity

The project meets the technical complexity requirement through its multi-component architecture:

- **Distinct Subsystems:** The Frontend (React/UI principles), Backend (REST API/DB management), and ML Microservice (Computer Vision/AI principles) each utilize distinct engineering disciplines.
- **Challenging Requirements:** Integrating multiple pre-trained AI models (MiDaS, SAM2, Stable Diffusion) into a coherent pipeline that handles image preprocessing, API orchestration, and delivers a photorealistic output is a significant technical challenge that matches the complexity of modern web applications.

4.2 DESIGN EXPLORATION

4.2.1 Design Decisions

- **Microservices Architecture:** Decoupling the ML components from the main backend.
 - **Rationale:** Allows the Python-based AI service to be developed, scaled, and potentially even hosted independently of the Node.js backend, improving maintainability.
- **Use of Replicate API vs. Self-Hosted Stable Diffusion:**
 - **Rationale:** Replicate abstracts away the immense complexity and computational cost of managing GPU instances and model versions, allowing the team to focus on application logic rather than AI infrastructure.
- **React-Konva for the Canvas:** Choosing a dedicated canvas library over a generic HTML5 canvas implementation.
 - **Rationale:** React-Konva provides high-level abstractions for shapes, images, and transformations, significantly accelerating the development of the complex moodboard editor with features like drag, resize, and rotate.

4.2.2 Ideation

Frontend Canvas:

- Option 1: Native HTML5 Canvas with custom JavaScript
- Option 2: SVG Manipulation with D3.js or similar
- Option 3: Library fabric.js.
- Option 4: Library react-konva (chosen)
- Option 5: Library p5.js for creative coding

4.2.3 Decision-Making and Trade-Off

For the frontend canvas implementation, we compared five options: native HTML5 Canvas, SVG manipulation (D3.js), Fabric.js, React-Konva, and p5.js. Each was evaluated on four criteria: ease of integration with React, performance for complex interactions, available features, and community support.

Option	Ease of	Performance	Features	Community/	Overall
--------	---------	-------------	----------	------------	---------

	Integration		(drag/resize/ rotate)	Docs	
HTML5 Canvas	Low	High	Low (manually implemented)	High	Medium
SVG (D3.js)	Medium	Medium	Medium	Medium	Medium
Fabric.js	Medium	Medium	High	Medium	High
React-Konva	High	High	High	High	Very High (chosen)
p5.js	Medium	Low	Medium	High	Medium

Trade-Offs:

- HTML5 Canvas offers maximum flexibility but requires building interaction logic from scratch, which increases development time.
- D3.js and SVG-based approaches are strong for data visualization but less suited for interactive image manipulation.
- Fabric.js provides many features, but has less seamless React integration.
- p5.js is oriented toward creative coding and prototyping, but lacks performance optimization for production use.
- React-Konva balances performance, React integration, and feature support, making it the most efficient choice for a production-ready moodboard editor.

Decision:

We selected **React-Konva** because it minimizes development overhead while providing the functionality required for resizing, rotating, and layering assets on the moodboard. The trade-off is some added dependency weight, but this is outweighed by the time savings and robust feature set.

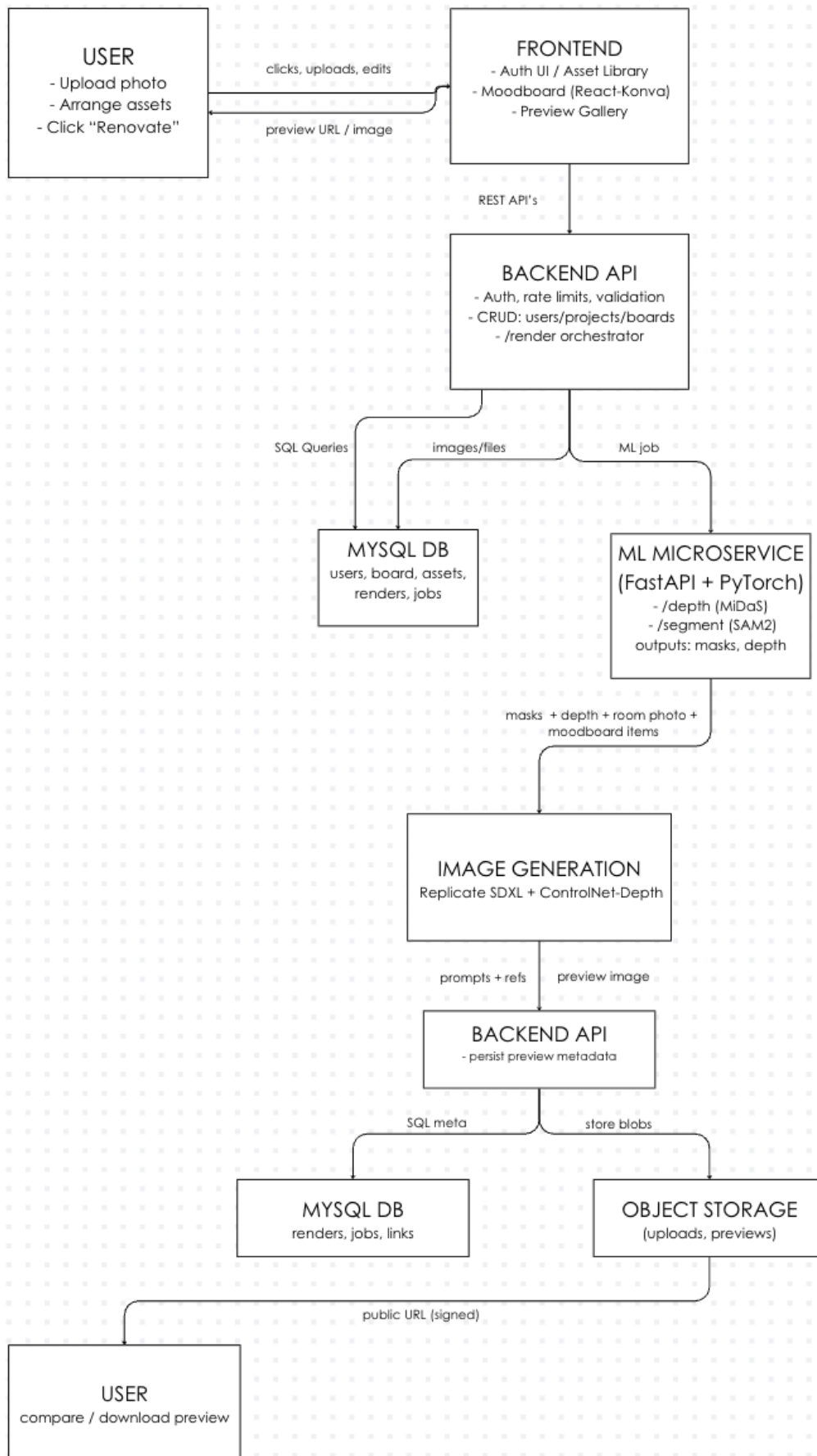
4.3 PROPOSED DESIGN

4.3.1 Overview

A user visits the Renova website. They upload a picture of their room and then pictures of furniture they like. They drag and drop that furniture onto the picture of their room, arranging it just how they want. When they click “Renovate,” our system works in the background to intelligently blend their choices into a brand-new, realistic picture of their redesigned room. The user can then compare the new version with the old one.

4.3.2 Detailed Design and Visual(s)

The Renova system is composed of several interconnected subsystems: the Frontend, the Backend API, the Database, the Object Storage layer, the Machine Learning Microservice, and the External Image Generation service. Each subsystem has a distinct role, and together they enable users to upload, manipulate, and preview realistic interior design renderings. The figure below illustrates the full system architecture and data flow.



Subsystem Overview

1. User Interface (Frontend – Next.js + React-Konva)

- Provides the core user experience: uploading room/furniture images, arranging assets on the moodboard canvas, and viewing generated previews.
- Built with Next.js for modern web features (routing, server-side rendering) and React-Konva for canvas operations (drag, drop, rotate, resize).
- Communicates with the backend via REST API calls (authentication, project management, render requests).

2. Backend API (Node.js + Express)

- Acts as the central orchestrator. Handles authentication, request validation, and CRUD operations for projects, boards, and assets.
- Provides the /render endpoint that coordinates the workflow: receiving user requests, invoking the ML microservice, sending data to the external image generator, and persisting results.
- Ensures rate limits, security, and proper job management.

3. MySQL Database

- Stores all structured metadata: users, boards, assets, job references, and preview links.
- Ensures relationships between users and their projects are maintained, and supports queries for retrieving history and managing access.

4. Object Storage

- Holds large binary files such as uploaded room photos, asset images, and generated previews.
- Unlike the database, which stores metadata, object storage is optimized for scalable, durable file storage.
- Access to stored files is managed via signed URLs, allowing secure, temporary access without exposing backend credentials.

5. ML Microservice (FastAPI + PyTorch)

- Dedicated service for computer vision preprocessing.

- Runs MiDaS for depth map estimation and SAM2 for segmentation masks.
- Outputs masks and depth maps that are passed back to the backend, which then integrates them into the rendering pipeline.
- Decoupled from the main backend for modularity and scalability.

6. External Image Generation (Replicate - Stable Diffusion XL + ControlNet-Depth)

- Handles the final rendering of photorealistic previews.
- Takes as input the room image, segmentation masks, depth maps, and moodboard arrangement, then generates a new image aligned with the user's design choices.
- Using Replicate offloads GPU complexity and ensures access to up-to-date model versions.

Integration and Data Flow

- The **user** begins by uploading room photos and assets in the frontend.
- The **frontend** issues REST calls to the **backend API**, which saves metadata to the **MySQL database** and files to **object storage**.
- When the user requests a renovation, the **backend** sends jobs to the **ML microservice** to obtain segmentation masks and depth maps.
- These artifacts are combined with user-provided moodboard data and sent to the **Replicate service**, which produces a rendered preview.
- The **backend** persists the preview metadata in the database and stores the generated image in object storage.
- The **frontend** retrieves a **signed URL** from the backend and displays the preview to the user for comparison and download.

This architecture separates concerns clearly: structured data vs. unstructured media, orchestration vs. preprocessing, and internal vs. external compute. Such modularity allows the system to scale effectively and ensures each component can be maintained or swapped independently if requirements evolve.

4.3.3 Functionality

Renova functions as an interactive web app where users upload room photos, add décor assets, and arrange them on a moodboard canvas. When the user clicks “**Renovate**,” the system processes their inputs and returns a photorealistic preview.

Behind the scenes, the backend orchestrates preprocessing (depth maps and masks), calls the external image generation service, and stores results in object storage. The user then receives a secure preview URL, allowing them to compare the new design with the original photo and iterate further.

In practice, the interaction is simple: **upload** → **arrange** → **renovate** → **preview**.

4.3.4 Areas of Concern and Development

The current design is well aligned with the project requirements and user needs, but there are several concerns that we anticipate as development moves forward. One of the most significant challenges will be ensuring that the system generates previews quickly enough to meet the two-minute target. Because rendering involves multiple subsystems, including external AI services, performance tuning will be critical. Another concern is the accuracy of the segmentation model, since errors in object detection could reduce the realism of the final previews. Cost is also a consideration, as frequent calls to the Replicate API may introduce scalability issues if usage grows rapidly. On the user side, smooth performance of the moodboard editor is essential, particularly when many assets are added to the canvas.

To address these issues, the immediate plan is to introduce caching and usage quotas to help with both performance and cost management. A manual mask refinement feature will be implemented so that users can correct mistakes made by the segmentation model. Frontend optimizations, including performance tuning for React-Konva, will be explored to ensure the editing experience remains responsive. Finally, usability testing will guide further refinements and ensure that the system remains practical and accessible for the target users.

4.4 TECHNOLOGY CONSIDERATIONS

Renova combines several distinct technologies, each chosen to balance development speed, scalability, and user experience. The following highlights the major components with their strengths, weaknesses, and trade-offs:

- **Frontend (Next.js + React-Konva):** Provides a modern, responsive UI and a high-level canvas API for drag-and-drop interactions. This accelerates development but can introduce performance bottlenecks when rendering many assets simultaneously.
- **Backend (Node.js + MySQL + Object Storage):** Node.js and Express handle REST APIs and orchestration, while MySQL stores structured metadata (users, boards, asset info) and object storage keeps large image files. This separation improves scalability but requires careful consistency management between DB records and stored files.
- **Machine Learning Microservice (FastAPI + PyTorch):** Runs MiDaS and SAM2 models independently from the backend. This keeps the main API lightweight and allows experimentation with different models, at the cost of additional complexity in deployment and inter-service communication.

- **External AI Generation (Replicate API with Stable Diffusion XL + ControlNet):**
Offloads GPU-intensive rendering to a third-party service, removing infrastructure overhead. The trade-off is recurring cost and partial dependence on an external provider; a self-hosted model remains a future alternative.

These technologies together create a modular, scalable system that supports a fast MVP. In the future, we may optimize or replace individual pieces (e.g., self-hosting models, performance tuning the canvas) to reduce costs and improve control.

4.5 DESIGN ANALYSIS

The project foundations have been established with initial frontend and backend repositories pushed to GitLab. Issues were created ahead of time, giving the team a clear backlog to work from as development begins. Parallel to this, the UI is being designed in Figma, focusing on the layout of the moodboard editor and preview gallery.

The immediate plan is to build out core MVP functionality, starting with user authentication, file uploads, and basic canvas interactions, supported by backend endpoints. This early work will serve to validate the architecture and guide adjustments as integration proceeds. Overall, the design remains feasible with the chosen technologies, and upcoming development will be directed at proving out the pipeline in practice.

5 Testing

Testing will be carried out continuously throughout development, beginning early and expanding as more features are implemented. Our approach connects directly to the project requirements: every functional requirement will have corresponding unit, integration, or system-level tests, while non-functional requirements such as performance and usability will be validated through user testing and acceptance testing. Because our system involves multiple subsystems (frontend, backend, ML microservice, external API), special attention will be given to integration testing of the /render pipeline. Automated testing will be run on every code change to catch regressions early, and usability testing with peers will ensure that the final product meets user expectations. Our upcoming lightning talk in November will focus specifically on testing, where we will outline this strategy and its role in validating the system.

5.1 UNIT TESTING

- Units: React components, Express route handlers, and helper functions in the ML microservice.
- Tools: Jest + React Testing Library (frontend), Jest + Supertest (backend), and Pytest (ML service).

5.2 INTERFACE TESTING

- Interfaces: API calls between frontend ↔ backend, and backend ↔ ML service.

- Approach: Verify that requests are correctly formatted and responses (success and error cases) are handled properly.
- Tools: Supertest and Postman.

5.3 INTEGRATION TESTING

- Critical Path: The /render job flow - Frontend → Backend → ML Service → Replicate API → Backend → Frontend.
- Justification: This path represents the core functionality; if it fails, the product cannot deliver previews.
- Tools: Cypress end-to-end testing, with the Replicate API mocked for reliability.

5.4 SYSTEM TESTING

- Strategy: End-to-end scenarios that replicate real usage, such as a user uploading a room image, adding furniture, and generating a preview.
- Connection to Requirements: Each test case will be mapped to specific functional requirements to ensure full coverage.
- Tools: Cypress.

5.5 REGRESSION TESTING

- Approach: Automated test suites (unit, integration, and end-to-end) will run via GitHub Actions on every pull request.
- Critical Features: Authentication, project save/load, and the image generation pipeline will be prioritized to ensure stability.

5.6 ACCEPTANCE TESTING

- Plan: Create a checklist of functional requirements and walk through each item in a live demo with clients and advisors.
- Client Involvement: Provide a pre-release build for independent use and feedback on whether the system meets expectations.

5.7 SECURITY TESTING (IF APPLICABLE)

- Passwords will be hashed and stored securely.
- Penetration-style tests (SQL injection attempts, malformed requests) will be used to validate API robustness.
- HTTPS and signed URL validation will be checked during deployment testing.

5.8 User Testing

- Plan: Recruit peers as test users and assign them tasks (e.g., “create a moodboard for this room”).
- Evaluation: Observe interactions and collect survey feedback on usability, intuitiveness, and overall satisfaction.

5.9 RESULTS

Testing has not yet begun, as the system is still in the setup and design phase. As implementation proceeds, results will be documented here to demonstrate compliance with requirements and highlight areas for refinement.

6 Implementation

Implementation is still in its early stages, with work focused on establishing the foundation for development. The frontend and backend repositories have been initialized and pushed to GitLab, and a backlog of issues has been created so tasks can be assigned as the team moves into feature development. On the design side, the user interface is being prototyped in Figma to guide implementation of the moodboard editor and preview gallery. These steps ensure that when development begins in full, the team will have both a technical framework and a clear visual reference to build from.

7 Ethics and Professional Responsibility

7.1 AREAS OF PROFESSIONAL RESPONSIBILITY/CODES OF ETHICS

Selected Code: ACM Code of Ethics (2018)

Area of Responsibility	Definition	Relevant ACM Code Item	How Our Team Applies It
Public Welfare & Safety	Build tools that help users and avoid foreseeable harm	1.1 Contribute to society; 1.2 Avoid harm	By providing safe previews of renovations before purchase, users avoid costly mistakes.
Honesty & Transparency	Accurately represent what the system can/can't do	1.3 Be honest and trustworthy	UI copy explains previews are approximations; errors are surfaced clearly.
Privacy & Data Protection	Protect user-uploaded photos and personal data	1.6 Respect privacy; 2.9 Design secure systems	Passwords hashed, files stored with signed URLs, HTTPS enforced.
Fairness & Inclusion	Ensure equitable access and avoid bias	1.4 Be fair and non-discriminatory	Templates cover diverse cultural styles; accessibility checks built into frontend.
Professional Competence	Stay within areas of expertise, seek review	2.2 Maintain professional competence	Code reviews, knowledge sharing, and use of established libraries/models.

Our team is performing well in Privacy & Data Protection. We have already planned secure password hashing, signed URLs for file access, and HTTPS enforcement to ensure user data is safe. This shows a strong commitment to user trust.

An area needing improvement is Fairness & Inclusion. Our current datasets and style presets may under-represent cultural diversity. To improve, we will expand templates, audit AI outputs for bias, and incorporate manual refinement tools for users to correct errors.

7.2 FOUR PRINCIPLES

Broader Context	Beneficence (do good)	Nonmaleficence (avoid harm)	Respect for Autonomy	Justice (fairness)
Public health, safety, welfare	Reduces stress and financial risk by letting users preview designs virtually	Clear disclaimers so users don't mistake previews for construction-ready blueprints	Allow users to delete accounts and export data on request	Free tier access ensures even small-budget users can benefit
Global, cultural, social	Style presets include global design traditions (e.g., Japanese minimalism, Mediterranean styles)	Avoid implying one cultural style is "default" or superior	Users select their own preferred aesthetics instead of being forced into defaults	Represent multiple cultural aesthetics equally in UI
Environmental	Helps reduce wasteful purchases, encouraging sustainable design choices	Avoid features that promote unnecessary consumption	Let users explore reuse/upcycle options when planning designs	Equal access to sustainable suggestions regardless of budget
Economic	Provides an affordable alternative to hiring designers	Ensure no hidden costs or dark patterns in pricing	Show cost per AI render clearly; allow cancellation of jobs	Transparent and consistent subscription/pricing for all users

Our team is strongest in Economic × Beneficence, where the system directly reduces renovation costs and stress by providing affordable previews before purchase. This demonstrates clear value to users.

An area for improvement is Global/Cultural × Nonmaleficence. Current datasets may under-represent non-Western design traditions, which risks biasing outputs. To address this, we plan to broaden training data, add culturally diverse templates, and allow manual adjustments in the UI.

7.3 VIRTUES

Team Virtues:

- **Integrity** – Be transparent about limitations of AI outputs and costs, so users trust the system.
- **Diligence** – Carefully test subsystems, validate AI results, and review code to maintain quality.
- **Collaboration** – Share knowledge across frontend, backend, and ML, ensuring all members contribute and learn.

Prakeerth Regunath

- **Demonstrated Virtue: Collaboration**
Collaboration has been a key component of senior design so far. It is important to me because I believe that no project truly succeeds without a team that can communicate effectively and support each other when they need it the most. I have demonstrated this virtue by actively participating and engaging in group discussions, sharing resources when needed, and actively agreeing with others.
- **Virtue Not Yet Demonstrated: Confidence**
Confidence is an important virtue to me because it allows many ideas to be shared clearly and taken seriously. Without confidence, even good contributions to the project may go unheard, which limits the value I can bring to the team. Sometimes, I tend to hold back my ideas or second-guess them before speaking up about them. In order to show more confidence, I plan to prepare thoroughly for the weekly meetings and trust the knowledge I myself gained, and voice my thoughts even if they aren't perfect. Doing this will help me grow personally and also benefit the team with fresh ideas and perspectives.

Onur Onal

- **Demonstrated Virtue: Integrity**
Integrity is important to me because it builds trust and keeps the team aligned. I have demonstrated this by being clear and transparent when defining project goals and delegating tasks, and by supporting teammates with both shared work and their individual responsibilities.
- **Virtue Not Yet Demonstrated: Patience**
Patience matters because technical progress can be slower than expected, and frustration can affect teamwork. I have not always shown this virtue yet, but I can demonstrate it by giving teammates the time and flexibility to work through challenges, while offering encouragement instead of pressure.

Bae Meh

- **Demonstrated Virtue: Patience**
Patience is important because it allows me to approach problems calmly and thoughtfully, rather than rushing and making mistakes. I have demonstrated patience when learning new tools and concepts for the project, as well as when tasks required extra time to understand fully. Instead of getting frustrated, I've stayed composed and worked through issues step by step.
- **Virtue Not Yet Demonstrated: Leadership**
and move toward the same goal. I can contribute to this by checking in with my teammates, providing additional input on the project's direction, and helping the group stay on track.

Erroll Barker

- **Demonstrated Virtue: Honesty**
Honesty/Accountability
Integrity has been important in my senior design work so far because, even during times with wedding issues, it has limited my ability to contribute. I was open and honest with our project leader (Onur) about my situation and took accountability for my inactivity, rather than making excuses.
- **Virtue Not Yet Demonstrated: Responsibility**
Responsibility
Patience and responsibility are important to me, and I have not demonstrated them as much as I would like. I am committed to taking on greater responsibilities and contributing more once I resolve my personal challenges related to my ongoing wedding issues.

Kemal Yavuz

- **Demonstrated Virtue: Collaboration**
Collaboration is important to me because it ensures that everyone contributes effectively and the team moves forward together. I have demonstrated this by actively engaging with both frontend and backend members, asking questions to clarify dependencies, and making sure my work connects smoothly with others. This has helped keep the team aligned across subsystems.
- **Virtue Not Yet Demonstrated: Diligence/Hard Work at Full Capacity**
Normally, I give 100% effort and push myself to deliver at my peak, but I haven't fully locked into that "prime version" of myself yet in this project. This matters to me because consistent hard work is what ensures the team's long-term success. I plan to demonstrate it by becoming more disciplined with my weekly contributions, diving deeper into tasks, and holding myself accountable to reach the level of effort I know I can sustain.

Om Sanghvi

- **Demonstrated Virtue: Responsibility**
Responsibility has been a key virtue for me so far in senior design. It is important to me because I believe being accountable for both technical quality and user trust makes the project sustainable. I have demonstrated responsibility by making sure our backend and deployment choices emphasize secure handling of user data. For example, in our meetings,

I raised points about password protection and signed URL access, and I also suggested a feature that would allow multiple people to collaborate on the same project board, while keeping access controls in place. These contributions show how I try to think not just about building features, but also about protecting the customer experience.

- **Virtue Not Yet Demonstrated: Diligence**

Diligence is important to me because consistent, detail-oriented effort is what ensures that a system as complex as Renova works reliably. While I've contributed ideas and focused on security concerns, I haven't yet demonstrated the full level of diligence I expect from myself in areas like testing, documentation, and polishing deployment pipelines. I plan to demonstrate diligence by carefully reviewing backend code, validating infrastructure configurations, and putting extra effort into thorough testing. This will help strengthen the system's reliability and make sure our users can trust the product.

8 Closing Material

8.1 CONCLUSION

Thus far, the team has defined the problem, surveyed prior work, and designed a modular three-tier solution for Renova that separates interactive creation (moodboard UI) from AI preprocessing (depth/masks) and photorealistic rendering. The Semester-1 focus is an MVP that delivers authentication, uploads, a performant moodboard, and a reliable /render pipeline via Replicate. Key risks include latency, segmentation accuracy, and API cost; mitigations include caching, quotas, and a manual mask refinement tool. If AI quality or cost becomes a blocker, the system still provides value as a high-quality planning/moodboard tool while the AI pipeline iterates. Overall, the scope is feasible within two semesters and aligned with user needs for low-risk, visual decision-making.

8.2 REFERENCES

- [1] Morpholio, "Board App," Morpholio, 2025. [Online]. Available: <https://www.morpholioapps.com/board>. [Accessed: 30-Sep-2025].
- [2] Canva, "Online Design Platform," Canva, 2025. [Online]. Available: <https://www.canva.com>. [Accessed: 30-Sep-2025].
- [3] RoomGPT, "AI Room Redesign," RoomGPT, 2025. [Online]. Available: <https://www.roomgpt.io>. [Accessed: 30-Sep-2025].
- [4] ACM, ACM Code of Ethics and Professional Conduct, 2018. [Online]. Available: <https://www.acm.org/code-of-ethics>. [Accessed: 30-Sep-2025].
- [5] ISO/IEC, 27001:2022 Information Security Management Systems — Requirements, 2022.
- [6] IEEE, IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications (note: superseded by IEEE 29148-2018).
- [7] ISO/IEC, 25010:2011 Systems and Software Engineering — System and Software Quality Models, 2011.

- [8] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, 2022.
- [9] A. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” in *Proc. CVPR*, 2022.
- [10] “React-Konva Documentation,” 2025. [Online]. Available: <https://konvajs.org/docs/react>. [Accessed: 30-Sep-2025].
- [11] Replicate, “Hosted Models and API,” 2025. [Online]. Available: <https://replicate.com>. [Accessed: 30-Sep-2025].

8.3 APPENDICES

- Appendix A — System Architecture Diagram (Fig. 4.3.2.1): Full data flow among Frontend, Backend, ML Microservice, Replicate, MySQL, and Object Storage.
- Appendix B — Sprint Schedule (Tbl. 3.4.1): Semester-1/2 sprint plan with milestones and integrated deliverables.
- Appendix C — API Spec v0.1: Endpoints, payloads, response schemas, and error codes for auth, projects, assets, and /render.
- Appendix D — Database ER Diagram: Entities and relationships (Users, Projects, Boards, Assets, Jobs, Previews).
- Appendix E — Risk Register: Probability, impact, owner, mitigation, and current status for top risks (cost, latency, segmentation accuracy).
- Appendix F — User Testing Pack: Consent script, task list, post-task survey (e.g., SUS or 5-point custom), and rubrics.
- Appendix G — Accessibility Checklist: WCAG 2.2 quick checks (contrast ratios, keyboard navigation, focus order, ARIA landmarks).
- Appendix H — License Inventory: Third-party libraries/models and licenses (React-Konva, MiDaS, SAM2, SDXL/ControlNet, etc.), usage notes, and attribution plan.

9 Team

9.1 TEAM MEMBERS

- PRAKEERTH REGUNATH - PKREGU22@IASTATE.EDU
- ONUR ONAL - ONUR@IASTATE.EDU
- KEMAL YAVUZ - KYAVUZ@IASTATE.EDU
- ERROLL BARKER - ERROLLB@IASTATE.EDU
- BAE MEH - MBAE@IASTATE.EDU
- OM SANGHVI - OSANGHVI@IASTATE.EDU

9.2 REQUIRED SKILL SETS FOR YOUR PROJECT

- Next.js
- React
- JavaScript
- Node.js
- Express
- REST APIs
- MySQL
- Schema Designs
- PyTorch
- FastAPI
- UI/UX
- Docker
- Vercel/AWS

9.3 SKILL SETS COVERED BY THE TEAM

- Next.js, React, JavaScript, UI/UX - Onur Onal, Prakeerth Regunath
- Node.js, Express, REST APIs - Onur Onal, Kemal Yavuz
- MySQL, Schema Designs - Erroll Barker, Bae Meh
- PyTorch, FastAPI, Docker - Bae Meh, Om Sanghvi
- Vercel/AWS (Deployment/Cloud) - Om Sanghvi, Onur Onal

9.4 PROJECT MANAGEMENT STYLE ADOPTED BY THE TEAM

The team has adopted an **Agile methodology with Scrum elements**, which supports iterative development and frequent reassessment of priorities. Agile was chosen because the project involves complex, interdependent features such as the moodboard editor, AI preprocessing pipeline, and integration with external services. Regular GitLab issue tracking, weekly in-person meetings, and ongoing communication via Discord allow the team to stay aligned, respond quickly to challenges, and deliver incremental progress throughout both semesters.

9.5 INITIAL PROJECT MANAGEMENT ROLES

- **Project Manager / Scrum Lead - Onur Onal** (coordinates sprints, oversees task tracking, ensures timely communication).
- **Frontend Lead - Prakeerth Regunath** (responsible for the moodboard editor, UI/UX design, and frontend integration).
- **Backend Lead - Erroll Barker** (responsible for Express APIs, authentication, and database integration).
- **Database & Schema Specialist - Kemal Yavuz** (designs and maintains MySQL schemas, ensures data consistency).
- **Machine Learning Lead - Bae Meh** (integrates SAM2, MiDaS, and PyTorch-based preprocessing in the FastAPI service).
- **Deployment & Infrastructure Lead - Om Sanghvi** (manages cloud deployment, Dockerization, and integration with Vercel/AWS).

9.6 Team Contract

Team Members:

- | | |
|------------------------------|----------------------|
| 1) <u>Prakeerth Regunath</u> | 2) <u>Bae Meh</u> |
| 3) <u>Erroll Barker</u> | 4) <u>Onur Onal</u> |
| 5) <u>Kemal Yavuz</u> | 6) <u>Om Sanghvi</u> |

Team Procedures

1. Day, time, and location (face-to-face or virtual) for regular team meetings:
 - a. Day of the week: Thursday
 - b. Time: 7 - 8 PM
 - c. Location: Mainly in-person. If there are issues, only then will we have a virtual (Discord) meeting, depending on availability.
2. Preferred method of communication updates, reminders, issues, and scheduling (e.g., e-mail, phone, app, face-to-face):
 - a. Primary: Discord (Group Chat + Meeting calls through phone/computer)
 - b. Secondary: Email (File-Sharing)
3. Decision-making policy (e.g., consensus, majority vote):
 - a. Majority vote when possible; otherwise, Consensus.
4. Procedures for record keeping (i.e., who will keep meeting minutes, how will minutes be shared/archived):
 - a. One rotating member will take meeting minutes each session.
 - b. Minutes will then be shared in our dedicated Discord group server

Participation Expectations

1. Expected individual attendance, punctuality, and participation at all team meetings:
 - a. Attend all scheduled meetings unless excused in advance.
 - b. Arrive on time and stay engaged during discussions.
2. Expected level of responsibility for fulfilling team assignments, timelines, and deadlines:
 - a. Each member is responsible for completing assigned work before deadlines.
 - b. If issues arise, notify the team early so adjustments can be made.
3. Expected level of communication with other team members:
 - a. Frequent and clear updates through Discord.
 - b. Respond to messages within 24 hours.
4. Expected level of commitment to team decisions and tasks:
 - a. Support group decisions once made.
 - b. Commit to high-quality work and follow through on tasks.

Leadership

1. Leadership roles for each team member (e.g., team organization, client interaction, individual component design, testing, etc.):
 - a. Prakeerth Regunath - Testing lead, ensures quality control and system verification.
 - b. Bae Meh - Documentation lead, keeps records updated and maintains shared files.
 - c. Kemal Yavuz - Team organizer, meeting coordinator, ensures deadlines are met.
 - d. Onur Onal - Design and architecture lead, coordinates technical design choices
 - e. Erroll Barker - Implementation lead, ensures coding standards and integration.
 - f. Om Sanghvi - Client/Instructor liaison, responsible for clarifying requirements
2. Strategies for supporting and guiding the work of all team members:
 - a. Pair programming or peer review for difficult tasks.
 - b. Offer help when someone falls behind.
 - c. Share resources, tutorials, and references to build up weaker skills.
 - d. Encourage open discussions to ask for clarification without judgment.
3. Strategies for recognizing the contributions of all team members:
 - a. Acknowledge contributions during meetings.
 - b. Post shoutouts in Discord for completed milestones.
 - c. Celebrate major project milestones together (e.g., small team social).

Collaboration and Inclusion

1. Describe the skills, expertise, and unique perspectives each team member brings to the team.
 - a. Prakeerth Regunath - Strong coding background and project management skills.
 - b. Bae Meh - Clear communicator, strong organizational abilities, effective at bridging ideas.
 - c. Erroll Barker - Detail-oriented, excels at documentation and tracking progress.
 - d. Onur Onal - Experience in frontend and backend development through internships. Came up with the project idea.
 - e. Kemal Yavuz - Good with system design and architecture, brings technical vision.
 - f. Om Sanghvi - Strong developer, skilled in integration and code quality assurance.
2. Strategies for encouraging and supporting contributions and ideas from all team members:
 - a. Encourage brainstorming without immediate criticism.
 - b. Use Discord polls for anonymous input on conflicting decisions.
 - c. Respect different working styles and schedules.
3. Procedures for identifying and resolving collaboration or inclusion issues (e.g., how will a team member inform the team that the team environment is obstructing their opportunity or ability to contribute?)
 - a. Team members should first raise the concern privately with the meeting organizer.
 - b. If unresolved, bring it up in a group meeting for open discussion.
 - c. If still unresolved, escalate the issue to the instructor/TA for mediation.

Goal-Setting, Planning, and Execution

1. Team goals for this semester:
 - a. Deliver a high-quality project on time by meeting the requirements, writing clean and maintainable code, and incorporating user feedback where possible.
 - b. Maintain consistent communication and collaboration to ensure transparency and alignment on project progress.
 - c. Ensure all team members understand and contribute to each part of the project so knowledge is shared and the workload is balanced.
2. Strategies for planning and assigning individual and team work:
 - a. Break large tasks into smaller pieces.
 - b. Assign based on individual strengths while rotating to build skills.
 - c. Use shared tools (Google Drive/Trello/Discord) for tracking progress.
3. Strategies for keeping on task:
 - a. Weekly check-ins on progress.
 - b. Clear deadlines and reminders in Discord.
 - c. Adjust workload distribution if someone falls behind.

Consequences for Not Adhering to Team Contract

1. How will you handle infractions of any of the obligations of this team contract?
 - a. First offense: Private reminder and discussion.
 - b. Second offense: Group discussion and redistribution of tasks if needed.
 - c. Third offense: Report to instructor/TA.
2. What will your team do if the infractions continue?
 - a. The team will document the issue and request intervention from course staff.

- a) I participated in formulating the standards, roles, and procedures as stated in this contract.
- b) I understand that I am obligated to abide by these terms and conditions.
- c) I understand that if I do not abide by these terms and conditions, I will suffer the consequences as stated in this contract.

1)	<u>Prakeerth Regunath</u>	DATE	<u>9/30/2025</u>
2)	<u>Bae Meh</u>	DATE	<u>9/30/2025</u>
3)	<u>Kemal Yavuz</u>	DATE	<u>9/30/2025</u>
4)	<u>Om Sanghvi</u>	DATE	<u>9/30/2025</u>
5)	<u>Onur Onal</u>	DATE	<u>9/30/2025</u>
6)	<u>Erroll Barker</u>	DATE	<u>9/30/2025</u>